

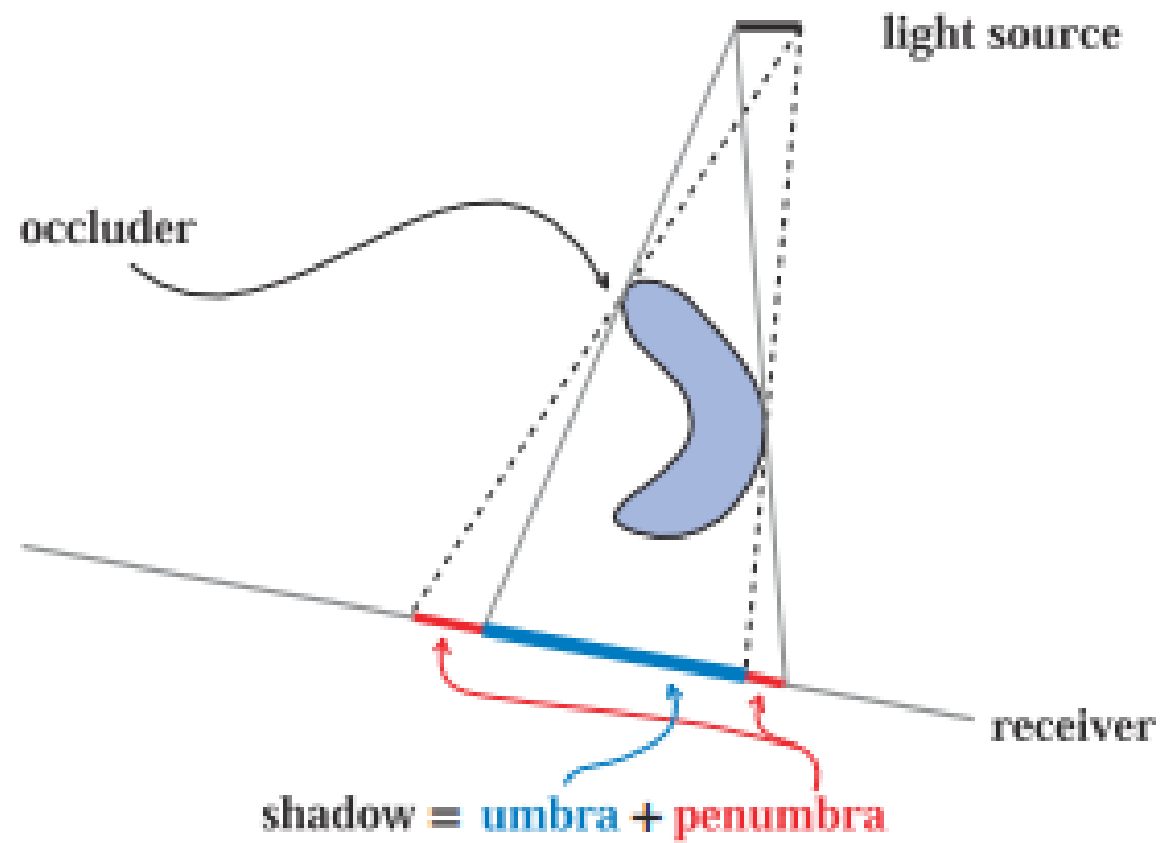


# 7장 Shadows

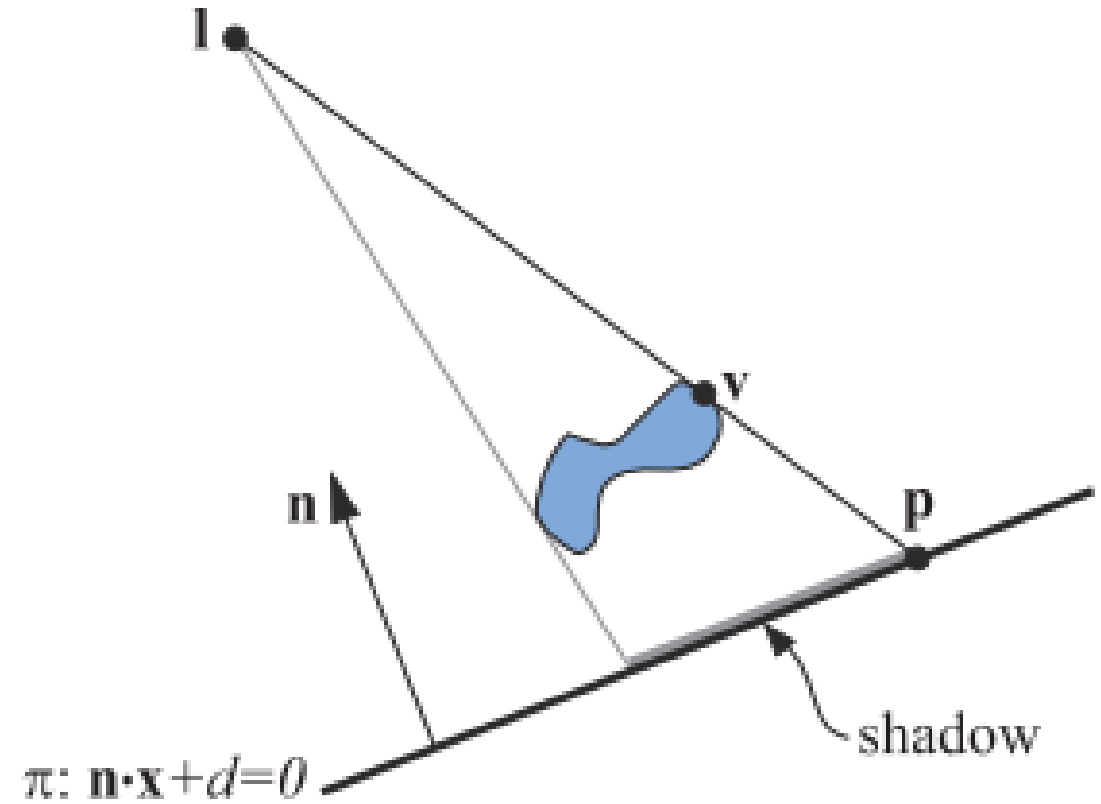
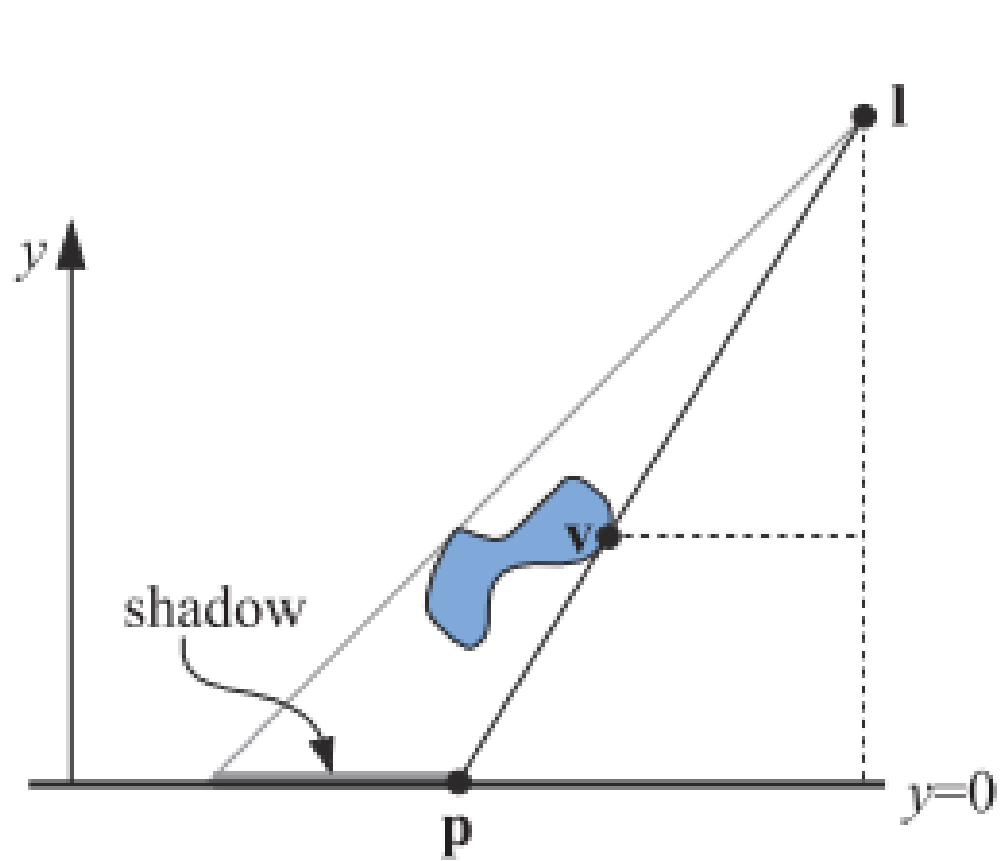
김해울

# 용어 정리

- **Occluders** cast shadows onto **receivers**.
- A **hard shadow** is produced when a punctual light source with no area is used. A **soft shadow** is produced when a light source with area is used.
- **Umbra** – a fully shadowed region
- **Penumbra** – a partially shadowed region



**Figure 7.1**



# Planar Shadows – Projection Shadows

Figure 7.3

Three-dimensional object is rendered a second time to create a shadow.

# Planar Shadows – Projection Shadows

- First, the projection matrix will be derived for the special case where the shadowed plane is  $y = 0$ , and then this result will be generalized to work with any plane.
- Left part of fig 7.3 – we get the following equation from similar triangles.
- $$\frac{p_x - l_x}{v_x - l_x} = \frac{l_y}{l_y - v_y} \Leftrightarrow p_x = \frac{l_y v_x - l_x v_y}{l_y - v_y}$$
- Z-coordinate is obtained in the same way. Y-coordinate is 0.
- These equations are converted into the projection matrix M. (A matrix such that  $Mv = p$ )

# Planar Shadows – Projection Shadows

$$\bullet \quad M = \begin{pmatrix} l_y & -l_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -l_z & l_y & 0 \\ 0 & -1 & 0 & l_y \end{pmatrix} \text{ for the special case.}$$

- In the general case, the plane the shadow is cast is not  $y = 0$ , but instead  $n \cdot x + d = 0$
- The ray which goes through  $v$  is intersected by the plane.
- The projected point  $p = l - \frac{d + n \cdot l}{n \cdot (v - l)} (v - l)$

# Planar Shadows – Projection Shadows

- Convert to projection matrix (7.4)

$$\begin{aligned}
 \bullet \text{ } M = & \begin{matrix} n \cdot l + d - l_x n_x & -l_x n_y & -l_x n_z & -l_x d \\ -l_y n_x & n \cdot l + d - l_y n_y & -l_y n_z & -l_y d \\ -l_z n_x & -l_z n_y & n \cdot l + d - l_z n_z & -l_z d \\ -n_x & -n_y & -n_z & n \cdot l \end{matrix}
 \end{aligned}$$

- Turns into the matrix from the special case if  $y = 0$ , because  $n = (0, 1, 0)$  and  $d = 0$ .
- Simply applying the above matrix to the objects that should cast shadows on the plane and the rendering it with a dark color and no illumination produces shadows.

# Planar Shadows

## – Projection

## Shadows

- Z-fighting : When the shadow is projected onto the plane, the shadow triangles and the ground plane exist at the same depth. Due to floating point errors, the GPU cannot determine which one is 'in front'.
  - Method 1 : Adding bias to the shadow
  - **Method 2** : Z-buffer manipulation through drawing order (**safer method**)
    - Z-buffer is a tool the GPU uses to keep track of how far away pixels are. Normally, it only draws a pixel if it is closer than what is already there.
1. Draw the ground.
  2. Disable Z-buffer. This tells the GPU to simply paint over whatever is already there without checking if the next object is deeper than the floor.
  3. Draw the shadow.
  4. Re-enable Z-buffer.



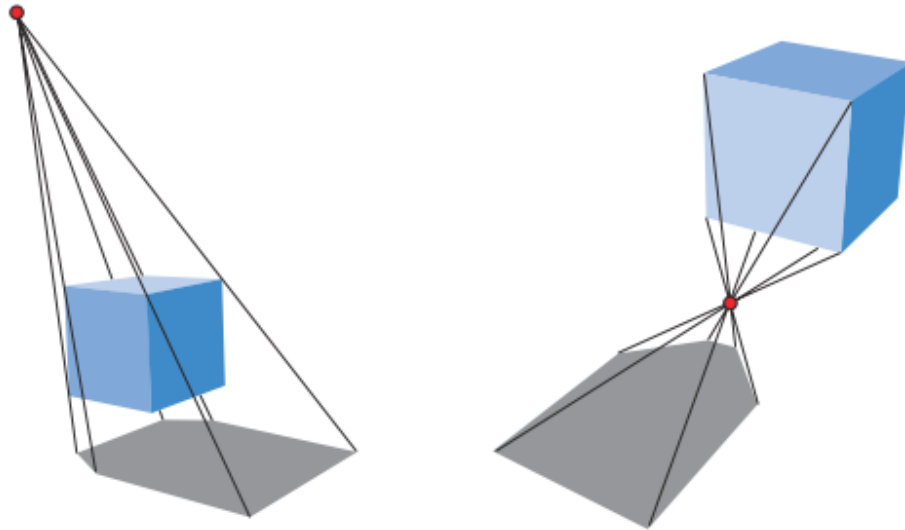
# Planar Shadows – Projection Shadows

- If the ground plane has a limit, and not infinite, the projection matrix method causes 'floating' shadows.
- Solution – The stencil buffer
- The stencil buffer is an offscreen buffer used to record the locations of the rendered primitive. (pg. 24)
- Draw the receiver to the screen and to the stencil buffer. Then, with the z-buffer off, draw the projected triangles only where the receiver was drawn.

# Planar Shadows – Projection Shadows

- Another approach – render the shadow triangles into a texture which is applied to the ground plane.
  - **Pros** : Allows penumbrae and shadows on curved surfaces.
  - **Cons** : The texture can become ‘magnified’ with a single texel covering multiple pixels.  
(Shadow looks blocky or jagged.)

# Planar Shadows – Projection Shadows



**Figure 7.4.** At the left, a correct shadow is shown, while in the figure on the right, an antishadow appears, since the light source is below the topmost vertex of the object.

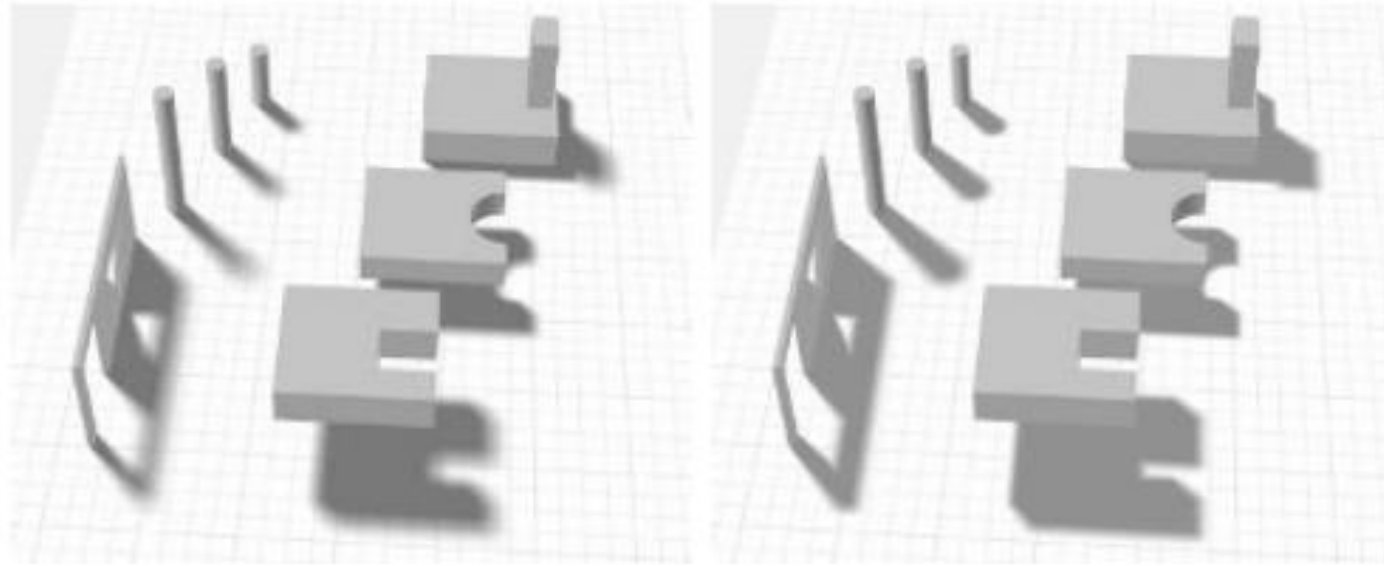
- Antishadow – Generated if the light source is below the topmost point on the object since each vertex is projected through the point of the light source.
- An error will also occur if we project an object that is below the receiving plane.

# Planar Shadows – Soft Shadows

- Heckbert and Herf's algorithm – Goal is to generate a texture on a ground plane that shows a soft shadow.
- The area light source is treated as **a collection of many punctual light sources**.
- A separate shadow is rendered for each punctual light source. The rendered shadows are accumulated and averaged.
- Where all shadows overlap, it is the **umbra**, and it is the **penumbra** where only some overlap.
- A frustum-based method – The idea is to treat the light as the viewer, and the ground plane forms the far clipping plane of the frustum.

# Planar Shadows – Soft Shadows

- Problem with the sampled area-light method – For  $n$  shadow passes, only  $n + 1$  distinct shades can be generated.
- Larger number of passes  $\Leftrightarrow$  Accurate result but excessive cost
- More efficient methods:
  - **Convolution.**
  - Blurring a hard shadow.
  - Haines' algorithm – starts with a projected hard shadow and then renders the silhouette edges to create plausible penumbrae.



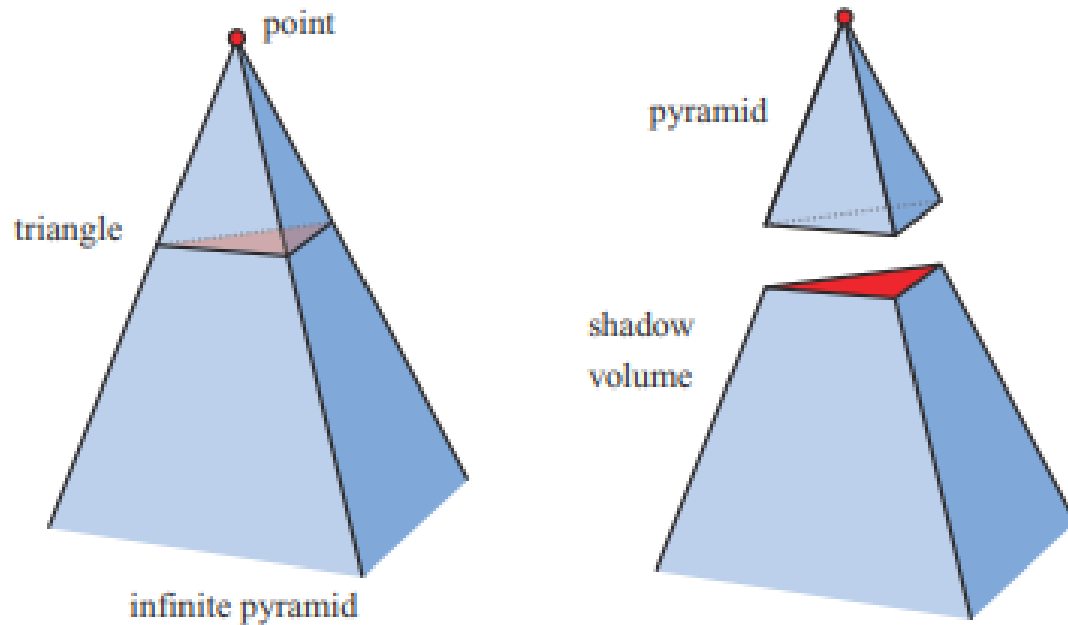
**Figure 7.5.** On the left, a rendering using Heckbert and Herf's method, using 256 passes. On the right, Haines' method in one pass. The umbrae are too large with Haines' method, which is particularly noticeable around the doorway and window.

## Planar Shadows – Soft Shadows

# Shadows on Curved Surfaces – Texture Projection

- What the light cannot 'see' is shadow.
- Occluder is rendered in black from the light's viewpoint into an otherwise white texture.
- The texture is then projected onto the surfaces.
- Each vertex on the receivers has a (u, v) texture coordinate computed for it and has the texture applied to it.
- **Drawbacks :**
  - Application must identify which objects are occluders and which are receivers.
  - Occluding objects cannot shadow themselves.

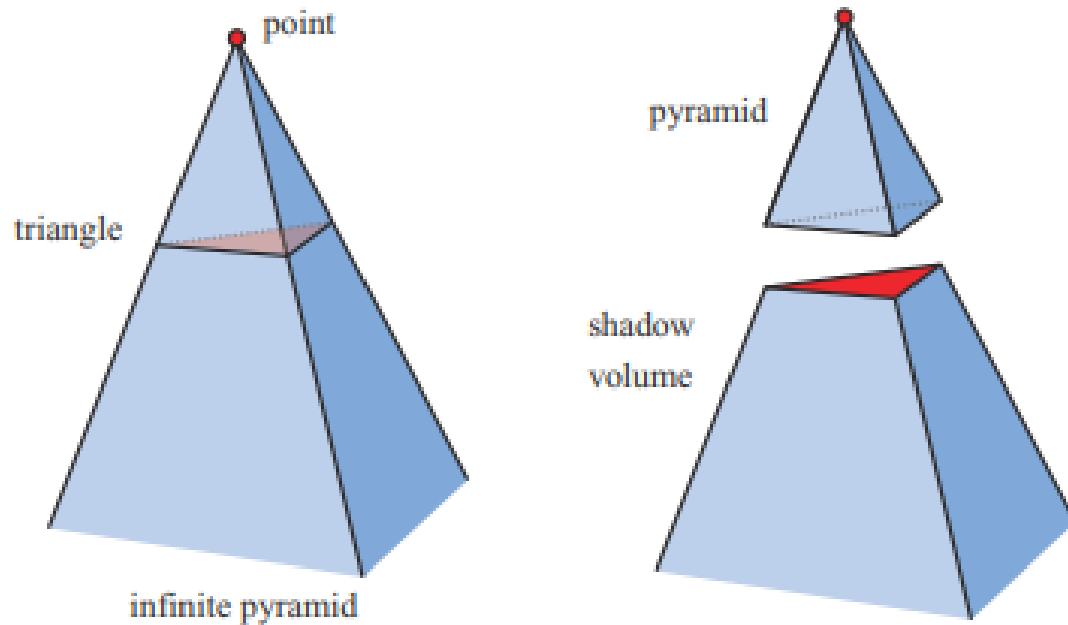
# Shadow Volumes



- Not image based thus avoiding sampling problems to produce correct sharp shadows everywhere. (Sometimes a disadvantage)
- Imagine following a ray from the eye through a pixel until the ray hits the object to be displayed.
- While the ray is on its way to the object, increment a counter each time it crosses a face of the shadow volume that is frontfacing.

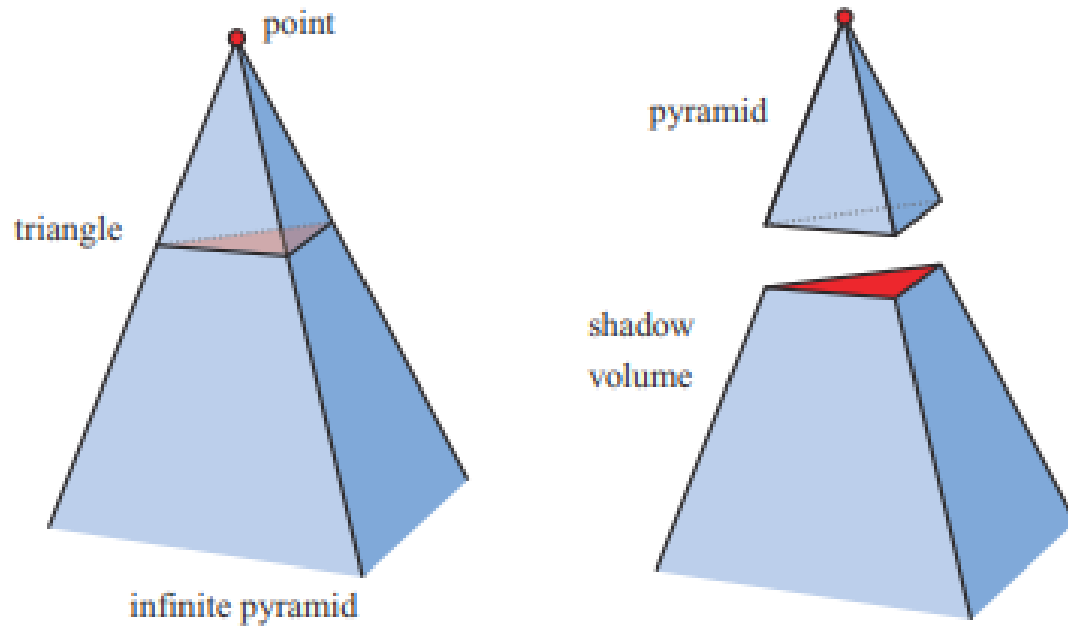


# Shadow Volumes



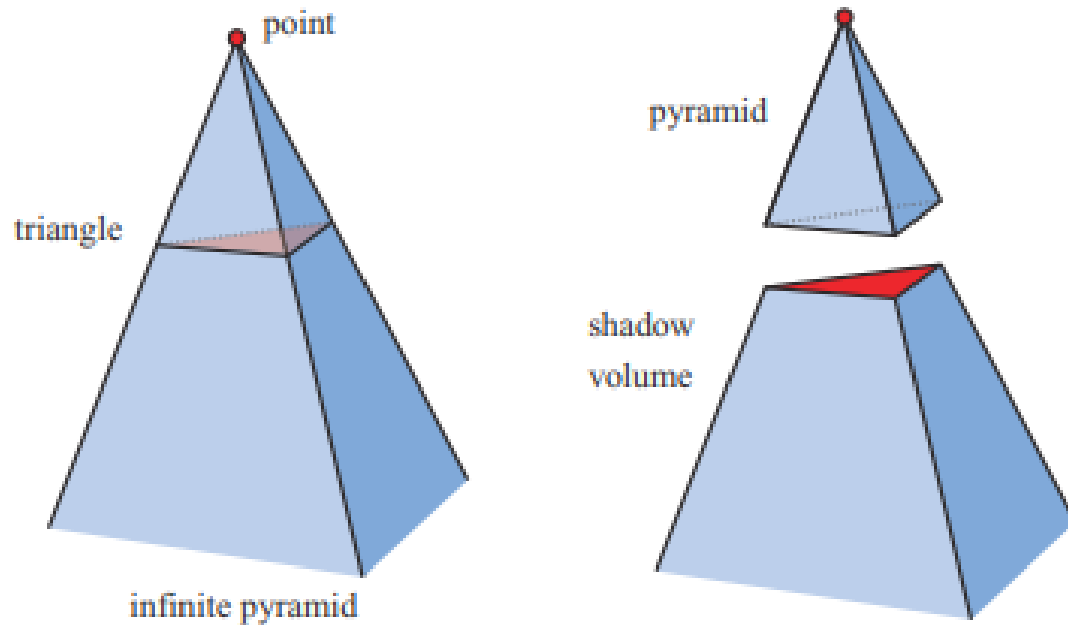
- Similarly, decrement the counter each time the ray crosses a backfacing face of the truncated pyramid. (Ray is going out of shadow)
- Increment and decrement counter until ray hits the object to be displayed at the pixel. If counter is greater than zero, the pixel is in shadow.

# Shadow Volumes



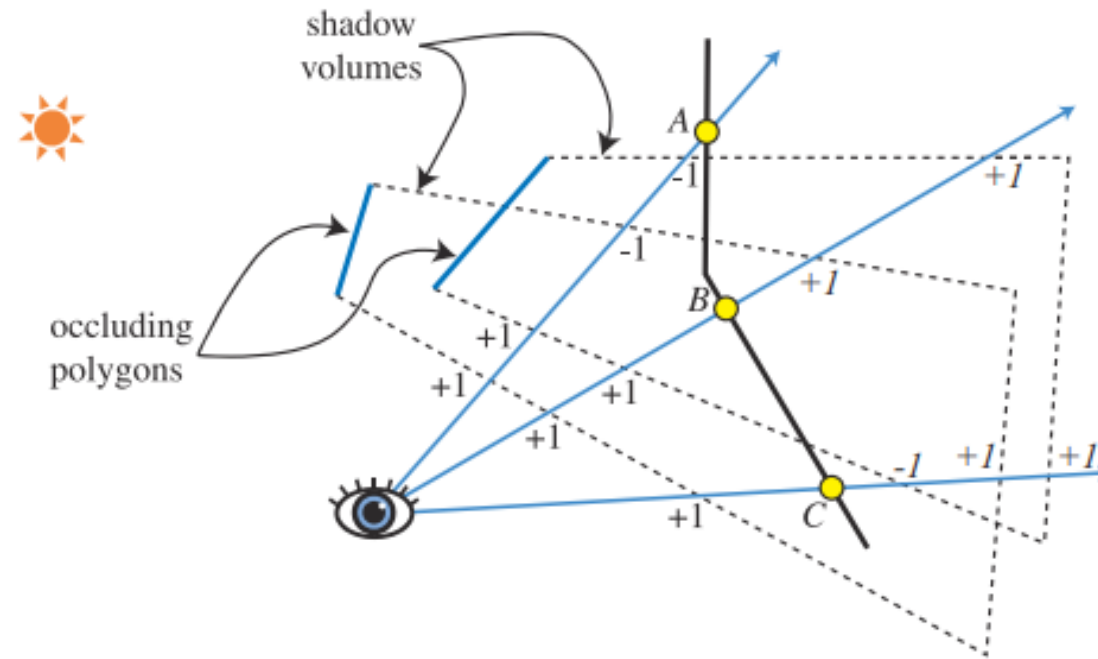
- Performing the procedure with actual rays is time consuming.
- Solution: Stencil buffer

# Shadow Volumes



1. The stencil buffer is cleared.
2. Whole scene is drawn into the framebuffer with only the color of the unlit material used.
3. Z-buffer updates and writing to the color buffer are turned off. Frontfacing triangles of shadow volumes are drawn. Stencil operation is set to increment the values in the stencil buffer wherever a triangle is drawn.
4. Another pass of the stencil buffer is carried out for the backfacing triangles. (Decrement counter. Counter is only updated when the pixels of the rendered shadow volume face are visible.)
5. Whole scene is rendered again with only the active materials that are affected by the light.

# Shadow Volumes



**Figure 7.8.** A two-dimensional side view of counting shadow-volume crossings using two different counting methods. In *z-pass* volume counting, the count is incremented as a ray passes through a frontfacing triangle of a shadow volume and decremented on leaving through a backfacing triangle. So, at point *A*, the ray enters two shadow volumes for *+2*, then leaves two volumes, leaving a net count of zero, so the point is in light. In *z-fail* volume counting, the count starts beyond the surface (these counts are shown in *italics*). For the ray at point *B*, the *z-pass* method gives a *+2* count by passing through two frontfacing triangles, and the *z-fail* gives the same count by passing through two backfacing triangles. Point *C* shows how *z-fail* shadow volumes must be capped. The ray starting from point *C* first hits a frontfacing triangle, giving *-1*. It then exits two shadow volumes (through their endcaps, necessary for this method to work properly), giving a net count of *+1*. The count is not zero, so the point is in shadow. Both methods always give the same count results for all points on the viewed surfaces.

# Shadow Volumes



**Figure 7.9.** Shadow volumes. On the left, a character casts a shadow. On the right, the extruded triangles of the model are shown. (*Images from Microsoft SDK [1208] sample "ShadowVolume."*)

# Shadow Volumes

- Drawback : Creating quadrilaterals for every triangle creates a huge amount of overdraw.
- Solution : Draw only quadrilaterals along the silhouette edges of the object.

# Shadow Volumes

- Another Drawback : **Extreme variability in computation**
- Makes it unusable in interactive applications where a consistent frame rate is important (such as online games).

# Shadow Maps

- In 1978, Williams proposed that a common z-buffer based renderer could be used to generate shadows quickly on arbitrary objects.
- Render the scene using the z-buffer, from the position of the light source that is to cast shadows. Each pixel in the z-buffer now contains the z-depth of the object closest to the light source. We call the entire contents of the z-buffer the **shadow map**.
- The scene is then rendered a second time with respect to the viewer. As each primitive is rendered, its location at each pixel is compared to the shadow map.
- If rendered point is...
  - Farther than the corresponding value in the shadow map, that point is in shadow.
  - Closer than the corresponding value in the shadow map, that point is not in shadow.



# Shadow Maps

- Solves the variability problem with shadow volumes.
- Cost is linear with the number of rendered primitives, and access time is constant.

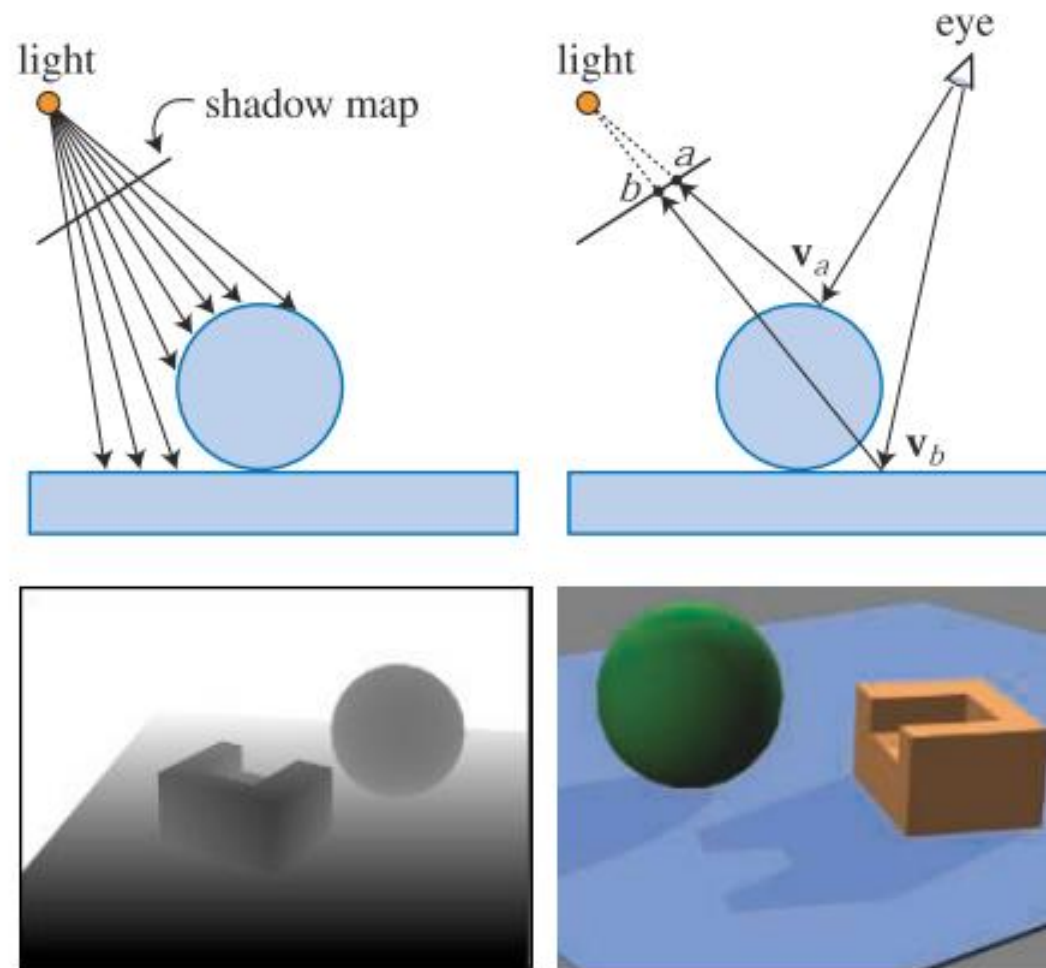
# Shadow Maps

- When a single z-buffer is generated, the light can only see in one direction just like a camera.
- For **directional light**, the view is set to encompass all objects casting shadows into the viewing volume that the eye sees. (Orthographic projection) The light's view needs to be made wide and high enough in x and y to view this set of objects.
- For **spotlight**, it naturally defines its own frustum based on its cone angle (perspective projection). Anything outside the cone isn't illuminated so the shadow map only needs to cover that specific area.

# Shadow Maps

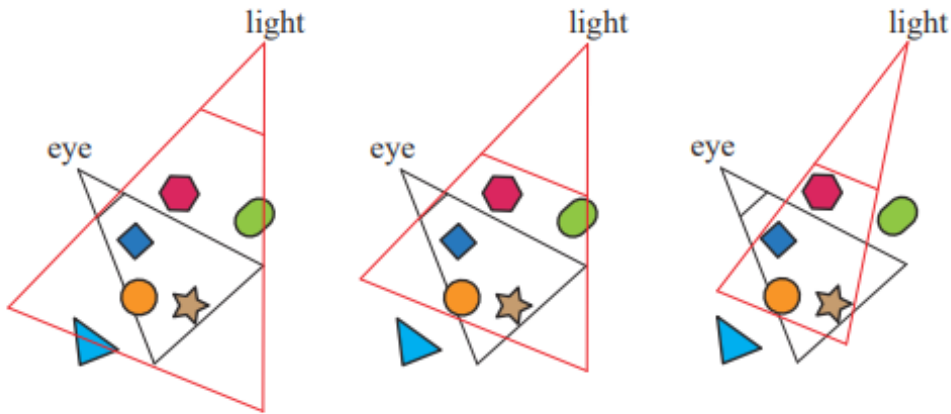
- **Problem case** : Local light source is inside a scene and is surrounded by shadow-casters.
- Light shines in all directions which cannot be covered by a single frustum.
- **Solution** : Use a six-view cube. (A cube map texture)
- Also called omnidirectional shadow maps.
- **Challenge** : Avoiding along the seams where two separate maps meet.
- Research has been conducted by many researchers, but their details are not introduced in this text. (King, Newhall, Gerasimov, Forsyth, Crytek)

# Shadow Maps



**Figure 7.10.** Shadow mapping. On the top left, a shadow map is formed by storing the depths to the surfaces in view. On the top right, the eye is shown looking at two locations. The sphere is seen at point  $v_a$ , and this point is found to be located at texel  $a$  on the shadow map. The depth stored there is not (much) less than point  $v_a$  is from the light, so the point is illuminated. The rectangle hit at point  $v_b$  is (much) farther away from the light than the depth stored at texel  $b$ , and so is in shadow. On the bottom left is the view of a scene from the light's perspective, with white being farther away. On the bottom right is the scene rendered with this shadow map.

# Shadow Maps



**Figure 7.11.** On the left, the light's view encompasses the eye's frustum. In the middle, the light's far plane is pulled in to include only visible receivers, so culling the triangle as a caster; the near plane is also adjusted. On the right, the light's frustum sides are made to bound the visible receivers, culling the green capsule.

- Not all objects need to be rendered into the shadow map just the shadow casters.
- The light's view frustrum can be augmented or tightened to disregard some shadow casters.
- Rendering only relevant objects not only can save time rendering but can also reduce the size required for the light's frustrum thus increasing quality.
- Effective precision of the z-buffer increases if...
  - The light frustrum's near plan is as far away from the light as possible.
  - The far plane is as close to the light as possible.

# Shadow Maps

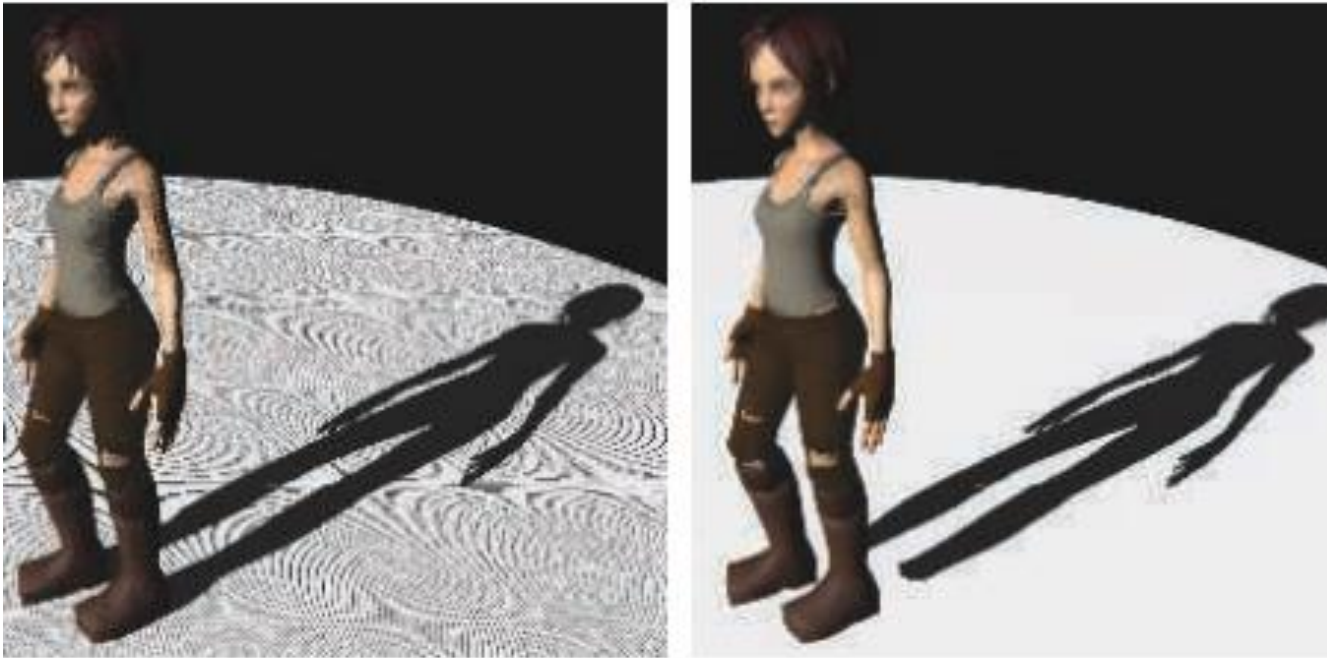


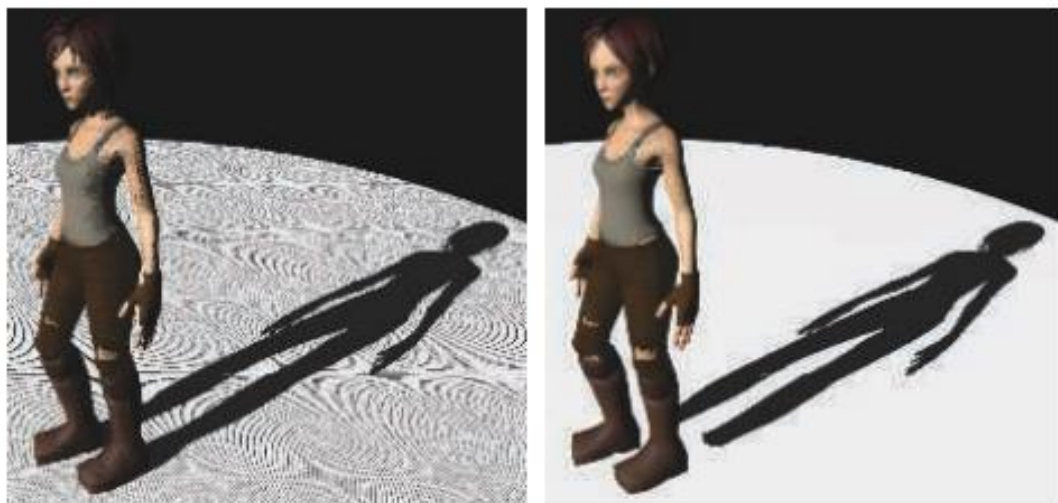
Figure 7.12. Shadow-mapping bias artifacts. On the left, the bias is too low, so self-shadowing occurs. On the right, a high bias causes the shoes to not cast contact shadows. The shadow-map resolution is also too low, giving the shadow a blocky appearance. (Images generated using a shadow denoiser by Christoph Peters.)

- A **disadvantage** of shadow mapping – The quality of the shadows depends on the resolution of the shadow map and on the numerical precision of the z-buffer.
- Also susceptible to aliasing problems.
- **Self-shadow aliasing** (also known as shadow acne) when a triangle is incorrectly considered to shadow itself.
  - Numerical limits of precision.
  - The value of a point sample is being used to represent an area's depth. (Samples generated for the light are almost never at the same locations as the screen samples)

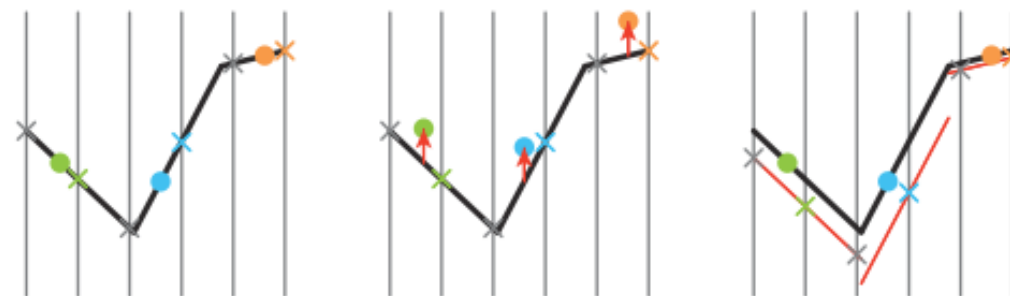
# Shadow Maps

- A remedy (but not a fix) is to introduce a bias factor.
- A small bias is subtracted from the receiver's distance when checking the distance found in the shadow map with the distance of the location being tested.
- **Constant** bias can fail when the receiver is not mostly facing the light.
- More effective – Use a bias that is proportional to the angle of the receiver to the light. (Slope scale bias) Often clamped to avoid extremely high tangent values.
- Both methods are used side by side to avoid possible precision errors.

# Shadow Maps



**Figure 7.12.** Shadow-mapping bias artifacts. On the left, the bias is too low, so self-shadowing occurs. On the right, a high bias causes the shoes to not cast contact shadows. The shadow-map resolution is also too low, giving the shadow a blocky appearance. (Images generated using a shadow demo by Christoph Peters.)



**Figure 7.13.** Shadow bias. The surfaces are rendered into a shadow map for an overhead light, with the vertical lines representing shadow-map pixel centers. Occluder depths are recorded at the  $\times$  locations. We want to know if the surface is lit at the three samples shown as dots. The closest shadow-map depth value for each is shown with the same color  $\times$ . On the left, if no bias is added, the blue and orange samples will be incorrectly determined to be in shadow, since they are farther from the light than their corresponding shadow-map depths. In the middle, a constant depth bias is subtracted from each sample, placing each closer to the light. The blue sample is still considered in shadow because it is not closer to the light than the shadow-map depth it is tested against. On the right, the shadow map is formed by moving each polygon away from the light proportional to its slope. All sample depths are now closer than their shadow-map depths, so all are lit.



# Shadow Maps

- Another solution to self-shadowing – Render only the backfaces to the shadow map.
- Second-depth shadow mapping.
- **Problem cases** : Object is two-sided, thin, or in contact with one another.
- Sousa et al. – Frontfaces work well for sun shadows and backfaces for interior lights.